## REA 定义

- REA : Reverse Engineer Anything

Understand how software works. Rebuild the feature you need.
理解软件如何工作。重建你需要的功能。

## 逆向环境

- node 26.13
- python 3.14
- ida-pro 9.5
- workbuddy 5.9.3
- ida-pro-mcp：https://github.com/mrexodia/ida-pro-mcp.git
- REA agent： https://github.com/morluto/rea

## 安装MCP

- workbuddy提示词：D:\REA\ida-pro-mcp-main.zip , 安装idalib-mcp 并验证可用性

**结果**

`idalib-mcp` 已安装好并通过真实反编译验证。

**安装位置**

- 虚拟环境：`D:\REA\idalib-mcp-venv`。它独立于系统 Python，不会污染全局环境。
- 源码：`D:\REA\ida-pro-mcp-main`，以 `pip install -e .` 可编辑方式安装。
- 入口：`D:\REA\idalib-mcp-venv\Scripts\idalib-mcp.exe`。

**关键点**：`idapro` 包不能用 PyPI 上的通用版本，必须安装 IDA 安装目录里那个本地 wheel，也就是 `C:\Program Files\IDA Professional 9.5\idalib\python\idapro-0.0.11-py3-none-any.whl`；否则它会找不到本机 IDA 内核。你的 idalib 之前已经激活过，`ida-config.json` 也指向了正确目录。

**验证结果**：用仓库自带的 `crackme03.elf` 做真实样本。

| 检查项 | 结果 |
| --- | --- |
| MCP 握手 | 通过，协议 2025-06-18 |
| 工具注册 | 66 个工具 |
| Hex-Rays 反编译 | 通过，拿到 `verify_password` 的完整 C 伪代码 |
| 交叉引用 | 通过，`main` 在 0x12d3 处调用 |
| 写操作（重命名） | 通过，并验证了持久化 |
| `idb_save` | 通过，生成 148KB 的 `.i64` |
| `analyze_function`、`survey_binary` | 通过，识别 28 个函数、17 个字符串 |

# 安装REA agent 代理

- 官网：https://reatool.dev
- github： https://github.com/morluto/rea
- workbuddy提示词： 使用npx rea-agents@latest setup安装REA , 并连接到这个编程助手，当前环境安装了idalib-mcp, 安装时启用idalib引擎，验证安装结果
- 重启workbuddy 连接器 信任开启 REA mcp



# REA 使用

- workbuddy提示词：使用 REA 分析 D:\REA\watchdog\sangforupdater.exe
- workbuddy提示词：根据 REA 的逆向分析，使用rust语言重写一个sangforupdater.exe ,生成后的文件保存在D:\REA\watchdog\rust 目录下

继续处理任务过程

这是一个 Go 1.26.4 编译的二进制文件，使用 `-H windowsgui` 构建且启用了 CGO，没有附带模块路径，说明源码被刻意剥离。Go 二进制会保留大量符号和字符串，因此我接下来从字符串入手。

继续处理任务过程

`inspect_binary_layout` 只支持 Linux ELF，因此改用字符串搜索。Go 二进制的关键信息就在符号和字符串里。

继续处理任务过程

只匹配到 1 条，我需要查看实际内容。现在改用 `inspect_analysis_view` 去读具体条目。

继续处理任务过程

关键发现：程序内嵌了一个下载 URL `https://app.zhuyan.cloud/v5/dl/cmd/sangfor.bin`。接下来继续查找其他 URL。

继续处理任务过程

search strings

```
{
  "parameters": {
    "document": "D:\\REA\\watchdog\\sangforupdater.exe",
    "detail": "summary",
    "mode": "regex",
    "pattern": "https?://[\\w.\\-]+"
  }
}
```

只有 3 条 URL。看看具体内容。

行为已完全定型。现在查看执行函数的细节，先从 `main.main` 开始，弄清楚完整调用链。

继续处理任务过程

IDA 还在跑二次分析。我先直接用已会话的 IDA 数据库读取，绕过 REA 的健康检查。

继续处理任务过程

worker 还在，但需要收养。我通过路径收养它。

继续处理任务过程

已经收养 REA 的分析数据库，直接反编译核心函数。

继续处理任务过程

执行链已完全清晰：`VirtualAlloc` → `memmove` → `VirtualProtect` → **`SetTimer` 回调执行** → 消息循环。这是经典的 shellcode 执行技巧。现在看主函数。

继续处理任务过程

decompile

深度思考

主逻辑是：先检查单实例，5 秒后触发主流程。现在确认这个延迟回调就是下载执行链。